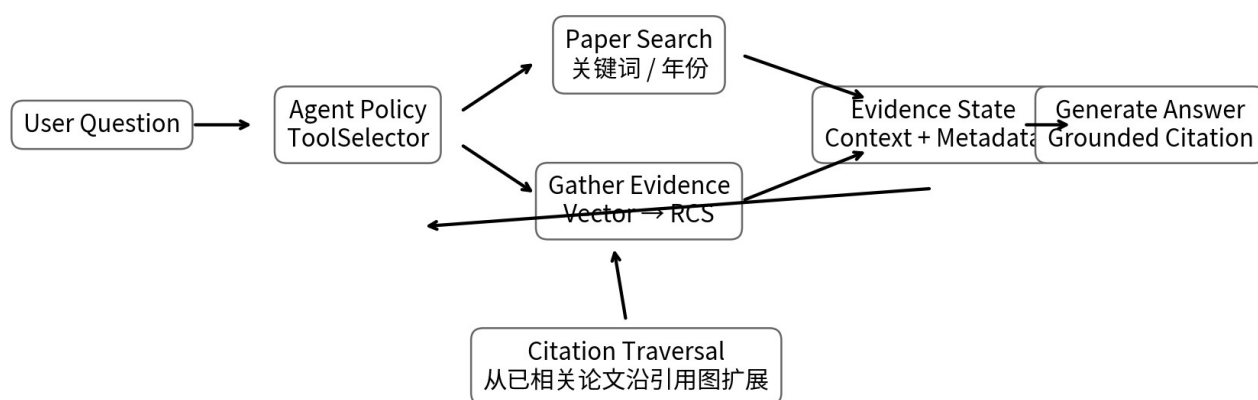


PaperQA2

Scientific Deep Research / Agentic RAG

完整技术研究 · Know-Why × Know-How × 论文 × 源码 × 实验



关键：Agent 不是一次性调用链，而是在 Paper Count / Relevant Papers / Evidence Count 状态上循环决策

核心架构：Agent 在论文检索、证据构建与答案生成之间循环决策

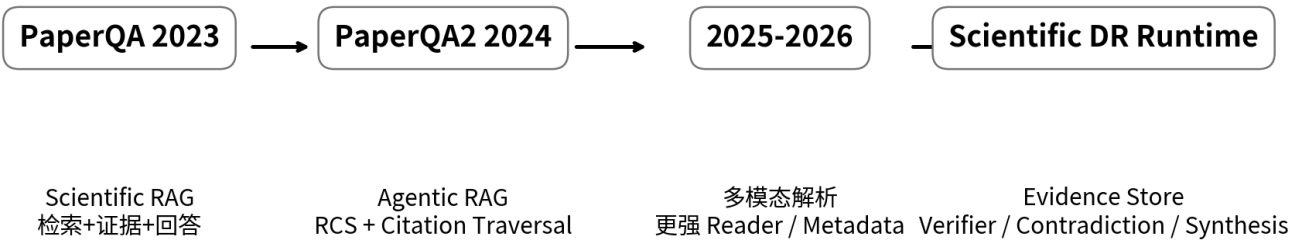
研究基线：PaperQA / PaperQA2 论文 + FutureHouse 工程博客 + Future-House/paper-qa 当前 main

源码快照与 Release：2026-08-13 / v2026.08.12

阅读导航与核心结论

一句话结论 PaperQA2 的本质不是“把更多论文塞给 LLM”，而是构造一个面向科学文献的 Evidence Operating System：Agent 主动找论文，RCS 把全文压成问题相关的证据单元，Citation Graph 扩展召回，最终只让高价值、可归因证据进入答案生成。

如果把通用 Deep Research 抽象成“开放世界搜索控制问题”，PaperQA2 更像“受科学文献结构约束的高精度证据控制问题”。它明确选择 Accuracy-first：愿意用更多模型调用和更深重排换取科学事实准确率。[S1][S2]



演进焦点：从“对给定 PDF 做 RAG” → “主动找论文、筛证据、校验归因、构建科学知识状态”

图 1 | PaperQA → PaperQA2 → Scientific Deep Research Runtime

最值得积累的 8 个 Know-How

- Search 与 Evidence Retrieval 必须分层：先找论文，再从已找到论文中找证据。
- 检索候选不应直接进最终 Context；应先做 query-conditioned contextual summarization。
- Agent 的价值主要来自“观察结果后重写搜索”，不是多一个角色名称。
- 科学检索应利用 Citation Graph，而不是只依赖 embedding 相似度。
- Metadata 是证据的一部分：来源、年份、引用量、期刊/撤稿状态都会影响可信度。
- 让系统能够明确返回 Insufficient Information，是高 precision 系统的重要能力。
- Evidence Store 应保留 claim ↔ passage ↔ paper 的 provenance，便于后续审计与矛盾检测。
- 科学 Deep Research 的真正产品形态不止 QA，还包括综述生成、矛盾发现、证据审计。

1. 为什么科学文献需要专门的 Deep Research 架构

普通 Web Search 的目标通常是找到“足以回答”的页面；科学研究则要求来源可验证、细节精确、引用可追溯，并经常需要穿过正文、补充材料、表格和引用网络。PaperQA2 论文指出，传统科学 QA 基准经常只给摘要、固定语料或直接给相关论文，因此不能代表真实科研检索。[S1]

PaperQA2 因此把问题定义为：面对整个科学文献空间，系统能否像研究者一样主动搜索、筛论文、阅读正文、构造证据，再生成可引用答案。

Know-Why：为什么不是“一次向量检索 + 长上下文”

- 正确论文本身可能难找：关键词可能与问题表述不同。
- 即使找到正确论文，真正答案可能只出现于正文的一小段。
- 加入太多 chunks 会引入 distractor，降低最终判断精度。
- 研究过程经常需要“先发现一个概念，再据此改写下一轮查询”。

设计原则 Scientific RAG 应优化 Evidence Recall × Evidence Precision，而不是单独优化 embedding recall。



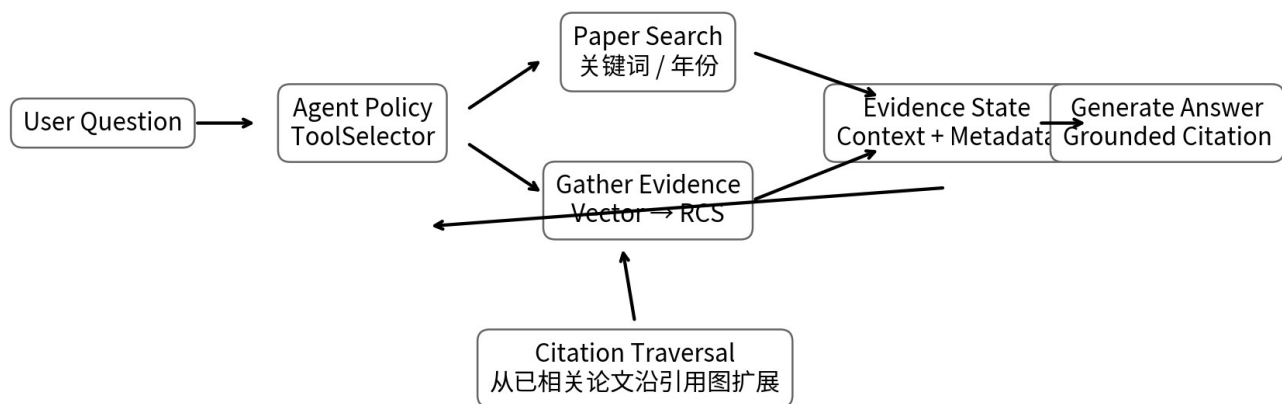
RCS 的目标不是“摘要所有文本”，而是把每个 chunk 转换成 Query-conditioned Evidence Unit：保留与问题相关的事实 + 来源元数据 + 相关性分数，过滤 distractor。

Know-how：先扩大候选集，再用昂贵模型做证据压缩；Context Window 留给高价值 evidence，而不是原始全文。

图 2 | RCS：把原始 chunk 转换成 Query-conditioned Evidence Unit

2. PaperQA2 核心 Agent Loop：Search → Gather → Answer → Research

论文版 PaperQA2 把 RAG 分解为工具；当前开源实现进一步把工具状态显式化。EnvironmentState 维护 Docs 与 PQASession，并向 Agent 暴露 Paper Count、Relevant Papers、Current Evidence、Current Cost 等状态。PaperSearch、GatherEvidence、GenerateAnswer、Reset、Complete 都是可调用工具。[S3][S4]



关键：Agent 不是一次性调用链，而是在 Paper Count / Relevant Papers / Evidence Count 状态上循环决策

图 3 | PaperQA2 当前 Agentic RAG 控制环

2.1 Paper Search：搜索的是“论文候选集”

当前源码中的 PaperSearch 接收 query 与年份范围，查询索引后把整篇候选论文的文本与元数据加入 state.docs。它还记录 previous_searches，相同 query 可以继续向后翻页；工具被标记为 CONCURRENCY_SAFE。[S4]

2.2 Gather Evidence：搜索的是“当前问题需要的证据”

GatherEvidence 会临时把 session.question 替换为更具体的问题，再调用 Docs.aget_evidence；完成后只向 Agent 显示当前 gather 问题的 top-n evidence，但完整 contexts 保存在 session 中。这是一种重要的 Context Engineering：Agent 看少量摘要，Runtime 保存完整证据状态。[S4]

2.3 Generate Answer / Reset / Complete

GenerateAnswer 使用当前 Evidence State 生成回答；工具描述明确建议至少有 5 条、来自多个来源的证据再回答。若证据污染，多次失败时 Agent 可 Reset 清空 contexts；Complete 则把“是否有成功答案”作为显式终止信号。[S4]

可迁移模式 把 Agent UI 状态设计成“研究进度仪表盘”：候选文档数、有效来源数、证据数、成本。模型更容易基于覆盖度决定继续搜还是停止。

3. RCS：PaperQA2 最关键的中间层

PaperQA2 先用 dense vector retrieval 获得较深的 Top-k 候选，再对每个 chunk 进行针对当前问题的 contextual summarization 与 relevance scoring，随后由 LLM 再排序，最终把最相关摘要放入 Evidence State。论文称这一过程为 RCS（reranking and contextual summarization）。[S1]

为什么 RCS 比“直接 Top-k chunks”更强？

- 压缩：把数百/数千 token 的段落压成只与问题相关的事实。
- 过滤：对无关 chunk 直接判低分，避免 distractor 进入最终 prompt。
- 标准化：不同论文、不同章节被转换为统一 Evidence Unit。
- 补充元数据：来源论文信息可以和证据一起进入判断。

论文消融显示，去掉 RCS 模型会显著降低 LitQA2 检索准确率；而小模型用于 RCS 甚至可能比“不做模型式 RCS”更差，说明 evidence compression 存在能力阈值。深度从 top-1 增加到 top-10 带来显著增益，10→30 仍有长尾收益，默认深度为 30。[S1]

Know-How “便宜 Embedding 扩召回 + 贵 LLM 压证据”通常比“让贵 LLM 一次阅读全部文本”更可控。

4. Citation Traversal：把引用图变成层级索引

仅靠语义搜索容易漏掉术语不同但互相引用的关键论文。PaperQA2 增加 Citation Traversal：从已发现的相关论文沿 citer / citee 关系扩展候选集。论文把它视为一种 hierarchical indexing，也指出其行为与科学家沿参考文献追溯/前向追踪的阅读方式一致。[S1]

LitQA2 高准确率运行平均每题约 1.26 次 paper search、0.46 次 citation traversal；这说明最优策略不是每题固定多轮，而是根据证据覆盖自适应追加搜索。[S1]

工程抽象 Scientific Search $\neq$ Text Search。更完整的检索图应包含 Semantic Edge + Citation Edge + Metadata Edge + Entity Edge。

5. LitQA2：为什么这个 Benchmark 对 Agent 有意义

LitQA2 包含 248 道需要从科学文献正文检索才能回答的多选题；问题刻意要求答案不在摘要中，并尽量让正确证据在整个文献中只出现一次。系统还可以选择“信息不足”，因此同时考察 precision、accuracy 和 DOI recall。[S1]

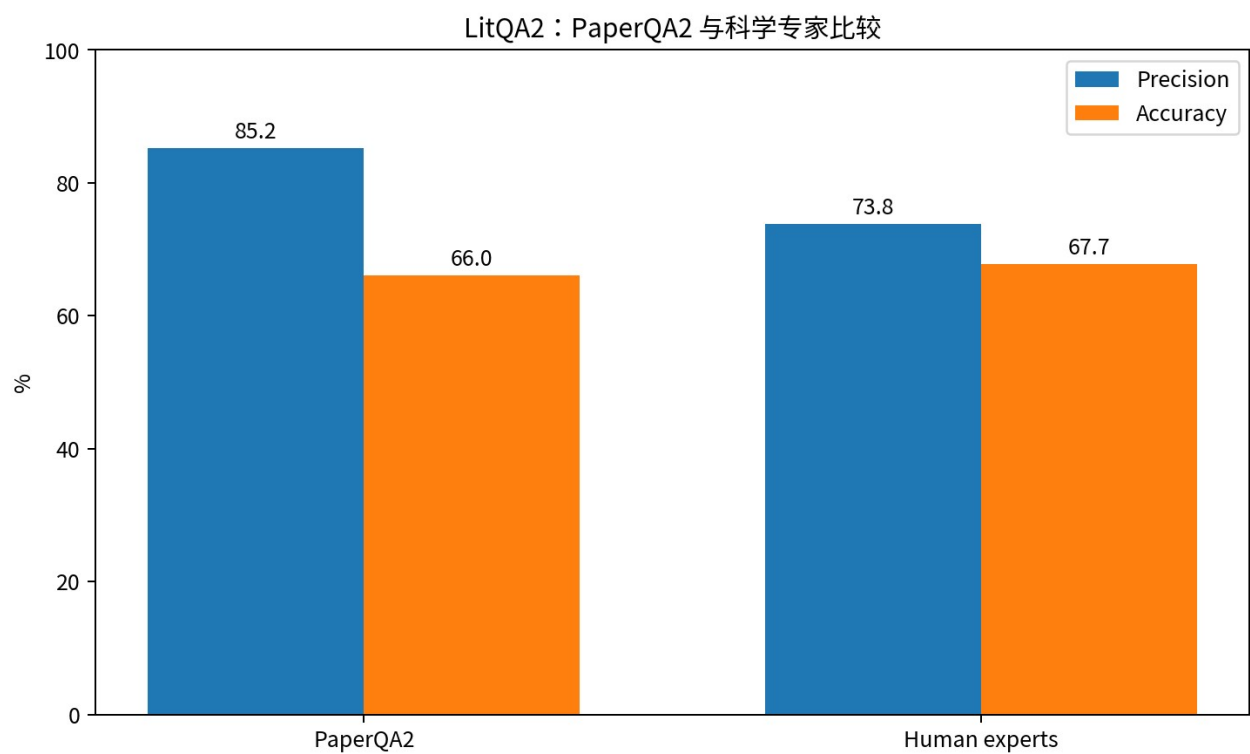


图 4 | LitQA2：PaperQA2 获得更高 Precision；Accuracy 与专家相当

PaperQA2 在 LitQA2 上达到 85.2% precision、66.0% accuracy；科学专家为 73.8% precision、67.7% accuracy。PaperQA2 约 21.9% 的问题选择“insufficient information”。平均每题解析约 14.5 篇论文。[S1]

Benchmark 的关键设计启示

- 正确答案必须依赖真实检索，而不是模型记忆。

- 用 DOI attribution 判断系统是否真的找到原始证据。
- 允许拒答，从而衡量 precision / coverage 的真实权衡。
- 保留新的 holdout questions，检验针对 benchmark 的工程优化是否泛化。

6. Ablation：真正决定准确率的是什么

组件	论文观察	工程含义
Agentic control	固定 Search→Gather→Answer 的 No-Agent 版本显著更差	允许模型根据证据状态回到搜索阶段
RCS	No-RCS 显著降低准确率；强模型优于弱模型	Evidence compression 是高能力模型值得花算力的位置
Top-k depth	1→10 显著提升；10→30 仍有长尾收益	先扩大 evidence candidate pool，再压缩
Citation Traversal	提升 DOI recall / 检索覆盖	加入非语义结构边
Context count	更多上下文提高找中证据概率，但也增加 distractor	最终 synthesis context 不应无限扩张
Parser quality	对 LitQA2 主指标影响有限，但表格抽取场景重要	不同任务的瓶颈不同，不应统一优化

核心结论 PaperQA2 的效果来自“多级漏斗”：Paper Recall → Chunk Recall → RCS Precision → Attribution → Final Synthesis。每一级都可以单独测。

7. 从 QA 到 Scientific Synthesis : WikiCrow

WikiCrow 用多个 PaperQA2 调用生成基因主题的不同章节，再由程序拼接和概览生成，形成带引用的 Wikipedia-style 科学综述。240 篇文章平均约 1219 词，平均生成时间约 491.5 秒，平均成本约 4.48 美元（论文 2024 年实验条件）。[S1]

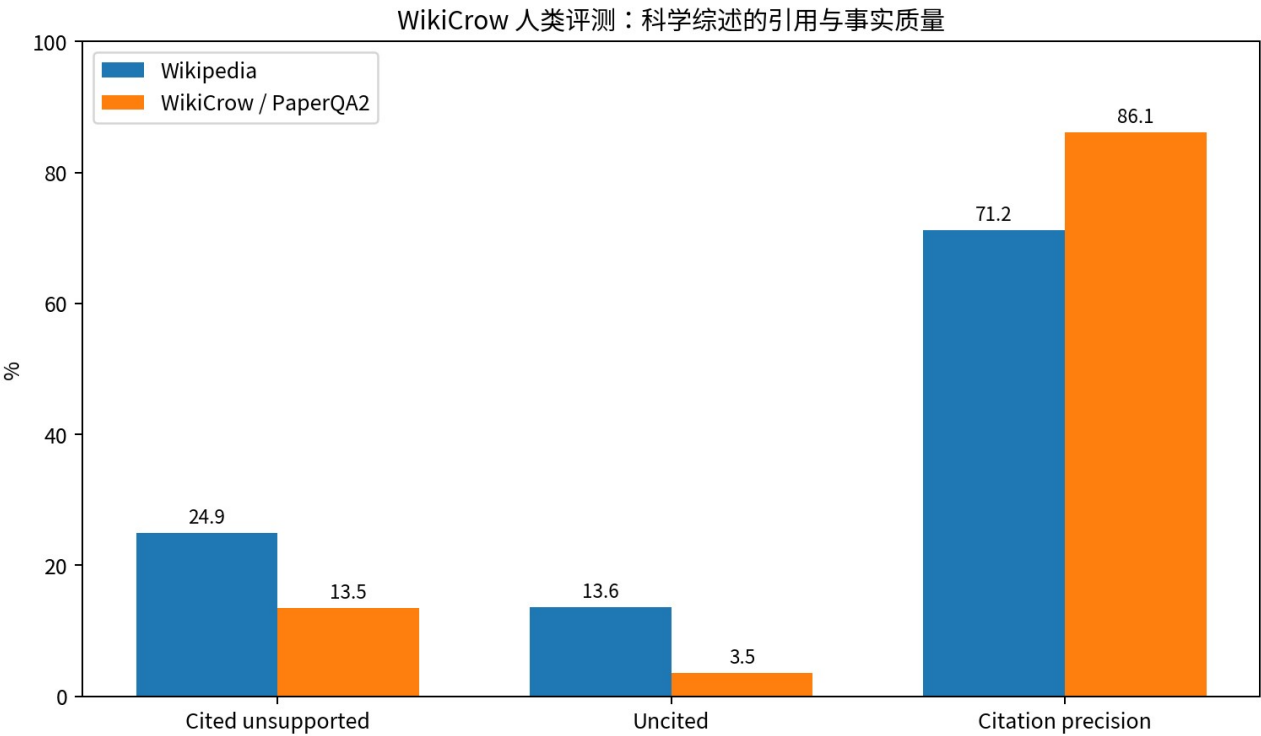


图 5 | WikiCrow vs Wikipedia：引用遗漏更少，引用精度更高

在人类盲评的 375 个陈述中，WikiCrow “有引用但不支持” 比例 13.5%，Wikipedia 为 24.9%；无引用陈述 3.5% vs 13.6%；引用 precision 86.1% vs 71.2%。误差分析显示 WikiCrow 的 reasoning errors 也少于 Wikipedia。[S1]

Know-Why 高质量科学综述不是单次 Prompt 问题，而是“多个可验证子问题 → 独立证据集合 → 结构化拼装”的 Artifact Pipeline。

8. 从回答到发现：ContraCrow

ContraCrow 把 PaperQA2 用在 “one-versus-many” 科学矛盾发现：先从目标论文抽取 claims，再为每条 claim 调用 PaperQA2 在整个文献中寻找支持/矛盾证据，并输出 11 点 Likert contradiction score。[S1]

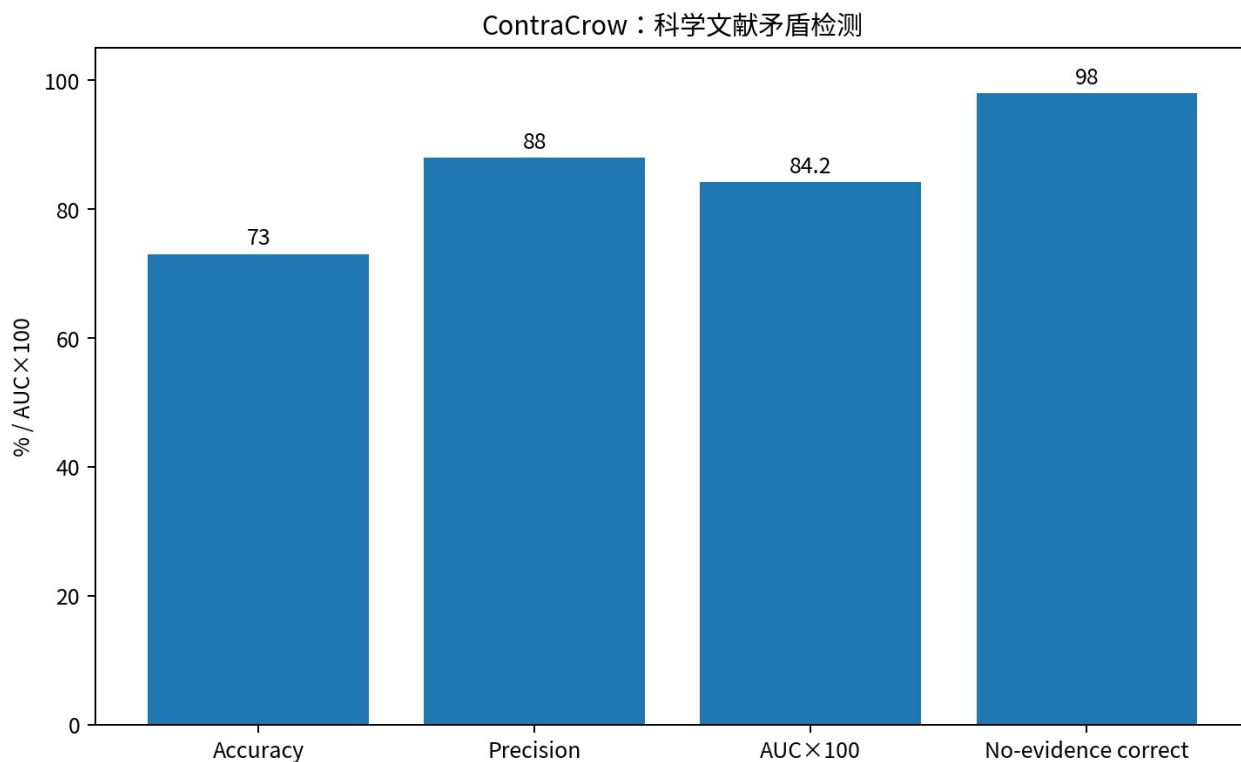


图 6 | ContraCrow 的主要验证指标

在由 LitQA2 转换的 ContraDetect 上，ContraCrow AUC 为 0.842；阈值 8 时 accuracy 73%、precision 88%、false positive rate 7%。对 42 个“从未报道”的 no-evidence statements，98% 正确选择缺乏证据。随后在 93 篇生物学论文中平均发现 2.34 ± 1.99 条矛盾/论文，其中专家验证约 70%。[S1]

产品化启发 Evidence Agent 一旦有可追溯的证据状态，就能从“回答问题”升级为“持续审计知识库”：检查 claim 是否过时、是否被后续研究推翻、是否缺少独立复现。

9. 2026 开源实现：从 PaperQA2 论文原型到 Scientific Runtime

截至 2026-08-13，Future-House/paper-qa 为 Apache-2.0 Python 项目，约 9,022 stars / 906 forks；最新 release 为 v2026.08.12。项目从 2025 年底采用 Calendar Versioning，PaperQA2 指 paper-qa>=5，而具体工程快照用日期版本表示。[S3][S5]

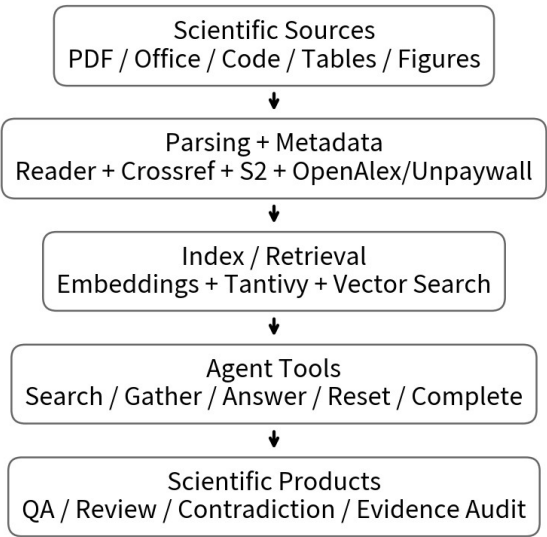


图 7 | 当前开源 PaperQA2 的工程分层

2025-2026 重要变化

- 支持 tables、figures、non-English、math equations 等多模态内容。
- 增加 Docling、NVIDIA nemotron-parse 等模型式 PDF reader；PDF reader 可报告页码、图表与 DPI。
- multimodal contextual summarization：媒体对象也参与 summary / enrichment。
- metadata pipeline 持续增强，对 Semantic Scholar / OpenAlex 等来源做冗余解析和修复。
- 工具层已经支持 clinical_trials_search 等垂直检索能力，表明架构可扩展到论文之外的科学数据库。

当前 PaperSearch 在本地/已构建索引上工作；README 也明确说明，论文实验内部拥有一些公开版本尚未完全具备的论文访问能力和 citation traversal 工具，因此开源复现实验时必须区分“算法能力”与“文献获取覆盖”。[S3]

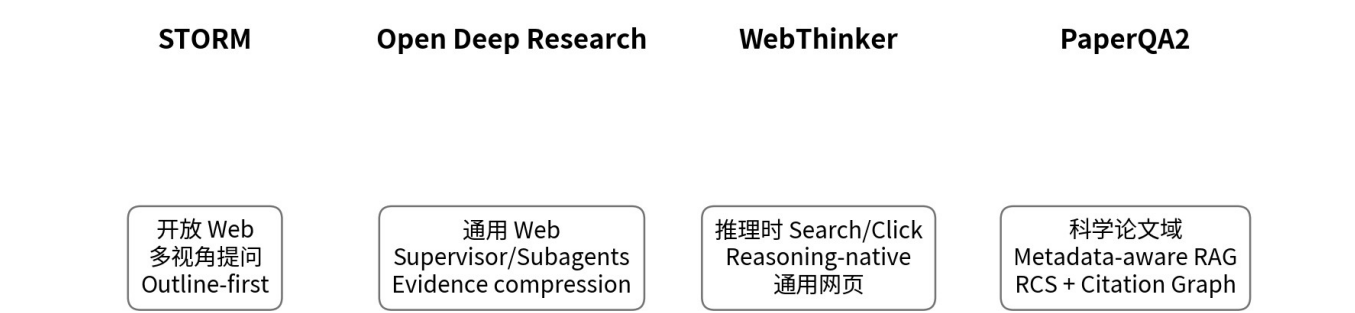
10. Metadata-aware RAG：科学 Agent 为什么要把元数据当一等信息

PaperQA2 会自动从多个 provider 获取论文元数据，并把来源信息注入检索/证据过程。科学领域中，标题/作者/年份/DOI/引用关系/期刊/撤稿状态并非装饰字段，而是判断证据可信度、时效性和重复来源的重要信号。[S2][S3]

可迁移的数据模型

对象	建议字段	用途
Document	DOI/title/year/authors/venue/retraction/citation_count	来源质量与唯一身份
Passage	doc_id/page/section/span/media_ids	精确归因与回看
Evidence	question/summary/score/source_passage	Query-conditioned evidence unit
Claim	text/evidence_ids/support_state	最终报告的 claim-level provenance
Relation	cites/cited_by/contradicts/supports	Evidence Graph

11. PaperQA2 与通用 Deep Research 的根本差异



PaperQA2 的优势来自“领域结构利用”：论文元数据、全文、引用图、来源质量与证据归因都成为一等对象。

图 8 | 不同 Deep Research 系统的主要结构偏置

系统	搜索空间	核心状态	主要优势	主要风险
STORM	Web	多视角问答/Outline	覆盖广、写作结构强	证据精度依赖检索质量
Open Deep Research	Web/MCP	Brief + subagent findings	动态分解与上下文隔离	长篇 evidence provenance 仍需强化
WebThinker	Web	Reasoning trajectory	搜索直接进入 reasoning policy	长轨迹成本/可审计性
PaperQA2	Scientific literature	Docs + Contexts + metadata	高精度 evidence + citation graph	论文访问覆盖、模型调用成本

PaperQA2 的关键结构偏置是“文献是一个有元数据、有引用边、有正文层级的图”，因此它能用比通用 Web Agent 更强的先验做检索和验证。

12. 可累积 Know-Why : 从 PaperQA2 学到什么

1. Agent 的价值是自适应检索

当第一次搜索结果不足时，模型能根据 evidence state 重写 query。把 agenticness 用在“下一步信息获取决策”，比用于角色扮演更有价值。

2. Context Engineering 的核心是 Evidence Transformation

原始网页/PDF 不是理想上下文。真正应进入主推理窗口的是被当前问题重新编码过的 Evidence Unit。

3. Recall 与 Precision 要分阶段优化

Paper Search 负责 recall，RCS 负责 precision，Final Context 再限制规模。不要用同一个 top-k 同时承担两类目标。

4. 图结构是检索的第二信号源

Citation Graph 可以发现语义表述不同但知识上相邻的来源。生产系统还可以加入作者、机构、实体、实验方法等边。

5. 拒答是 Research Agent 的能力

高可靠研究系统需要知道“证据不足”，而不是总要生成完整答案。

6. Benchmark 应测 provenance

只测答案文本会隐藏 lucky guess；应评估 source recall、citation precision、claim support。

13. 可累积 Know-How : 如何搭建一个 PaperQA2-style MVP

1. 建立 Document Store: PDF→page/section chunks，并保留 DOI、page、section、metadata。
2. 建立双层检索: Paper Search 先选文档，Evidence Retrieval 再选 chunks。
3. Evidence Retrieval 取较深 Top-k（例如 20-40），不要直接进入 final prompt。
4. 对每个 chunk 调 Summary/Reranker 生成 {summary, relevance, provenance}。
5. 维护 Evidence State；主 Agent 每轮只读取状态摘要和 top evidence。
6. 提供 Search / Gather / Answer / Reset / Complete 五类工具。
7. 加入 Citation/Reference Traversal，允许从高相关论文扩展邻居。
8. Final Writer 只读取高分 Evidence Units，并强制 claim-level citation。
9. 建立 stage-wise eval: paper recall → chunk recall → evidence precision → citation precision → answer quality。
10. 再扩展到 Review / Contradiction / Evidence Audit，而不是只做 QA。

推荐 Runtime 公式 Scientific Deep Research = Agent Policy × Scientific Search Graph × Evidence Compression × Provenance Store × Synthesis / Verifier

14. 下一代 Scientific Deep Research : PaperQA2 之后还缺什么

PaperQA2 已经把检索、证据压缩、归因和文献结构利用做得非常系统，但下一代系统还可以进一步把 Evidence State 从“上下文列表”升级成持久的 Evidence Graph。

- Claim-level verifier：生成报告后逐句检查每个 claim 是否被 evidence entail。
- Contradiction-aware synthesis：当来源相互冲突时，不做平均化总结，而是显式表示 disagreement。
- Temporal evidence：区分早期结论、后续修正与最新共识。
- Study-quality modeling：把样本量、研究类型、预注册、meta-analysis 等纳入 evidence weight。
- Retrieval-on-write：写每一节时从 Evidence Store 重新检索，而不是一次性把全部 evidence 塞进上下文。
- Cost-aware scheduler：根据不确定性决定是否继续 search/RCS/citation traversal。
- Persistent scientific memory：把已验证的 claim-evidence graph 作为下一次研究的可复用资产。

最终抽象 未来最强的科研 Agent 不应只是“会搜论文的聊天机器人”，而应是一个 Evidence-centric Scientific Runtime：搜索、阅读、归因、验证、冲突建模、写作共享同一个证据图。

结论

PaperQA2 的意义在于，它证明了 Deep Research 的可靠性可以通过系统工程被显著提高：Agent 负责控制搜索；RCS 负责把大规模文本变成高价值证据；Citation Graph 提高召回；显式 attribution 让答案可审计；同一底层系统还能扩展为 WikiCrow 与 ContraCrow。

对 Agent 工程而言，最可迁移的不是某个具体 Prompt，而是这个分层思想：不要让主模型同时承担搜索、全文阅读、相关性判断、上下文压缩和最终写作。把每一步变成可评测、可替换的 evidence transformation stage，才能让准确率随着算力与工具能力稳定增长。

附录 A | 源码阅读地图

文件	作用	优先级
src/paperqa/agents/tools.py	PaperSearch / GatherEvidence / GenerateAnswer / Reset / Complete 工具与 EnvironmentState	★★★★★
src/paperqa/agents/main.py	agent_query 与运行入口	★★★★★
src/paperqa/docs.py	aget_evidence / aquery，证据与答案核心	★★★★★
src/paperqa/agents/search.py	索引构建与 Paper Search	★★★★☆
src/paperqa/settings.py	RCS、evidence_k、source limits、agent 参数	★★★★☆
src/paperqa/readers.py	文档解析与多模态 reader	★★★☆☆
src/paperqa/prompts.py	Evidence summary / answer 等 prompt	★★★☆☆

附录 B | 主要来源

[S1] Skarlinski et al., Language agents achieve superhuman synthesis of scientific knowledge, arXiv:2409.13740 (2024).
<https://arxiv.org/abs/2409.13740>

[S2] FutureHouse Engineering Blog: Journey to superhuman performance on scientific tasks (2024-09-19).
<https://www.futurehouse.org/research/engineering-blog-journey-to-superhuman-performance-on-scientific-tasks>

[S3] Future-House/paper-qa README / current open-source PaperQA2.
<https://github.com/Future-House/paper-qa>

[S4] Current source: src/paperqa/agents/tools.py and related agent modules.
<https://github.com/Future-House/paper-qa/tree/main/src/paperqa/agents>

[S5] GitHub release v2026.08.12 and repository metadata snapshot, accessed 2026-08-13.
<https://github.com/Future-House/paper-qa/releases/tag/v2026.08.12>

[S6] Lála et al., PaperQA: Retrieval-Augmented Generative Agent for Scientific Research, arXiv:2312.07559 (2023).
<https://arxiv.org/abs/2312.07559>